DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA06 – Design and Analysis of Algorithms

Assignment 2 – CO2

Brute-Force and Exhaustive-Search Techniques

Final Report: Q1 – Q7

(Predict → Design → Implement → Measure → Analyse → Evaluate → Diagnose → Redesign → Validate)

Item	Detail
Student name	Shaik Shahid
Reg. No.	192424311
Programme	B.Tech – Artificial Intelligence and Data Science
Institution	SIMATS Engineering (Saveetha School of Engineering), Chennai
Assignment nature	Individual – 50 marks – 1 week
Implementation language	Python 3 (CPython 3.12.3) with matplotlib for graphs
Repository	CO2_Assignment/ (Q1 … Q7 folders: Pseudocode, Implementation_Code, Output_Screenshots; Final_Report; README.md)

Contents and Experimental Method

Section	Topic	Marks
Q1	Sorting under different data conditions	6
Q2	When is sequential search appropriate?	4
Q3	Behaviour of brute-force string matching	6
Q4	Closest pair and the cost of examining every possibility	6
Q5	Convex hull derived from a geometric principle	7
Q6	Exhaustive search and combinatorial growth	8
Q7	Day-4 Design Challenge – diagnose, redesign, validate (redesign of Q6)	13
–	Summary of predictions vs observations, limitations, reproducibility	–

Method used in every question

- **Predictions were written first.** For each question the pseudocode file (Pseudocode/Qn_pseudocode.md) contains a Predictions section written before the code was executed; this report quotes those predictions verbatim and then compares them with the measured results, including where the prediction was wrong.
- **Operation counts are the primary evidence** (comparisons, movements, pairs, distance calculations, side tests, candidates, nodes). Execution time is reported as supporting evidence only.
- **Reproducibility:** every random data set uses a fixed seed; raw results are stored in Implementation_Code/results (CSV/JSON and the captured console output).
- **Environment:** Linux sandbox, CPython 3.12.3, single-threaded, time.perf_counter(). Absolute times depend on the machine; the growth patterns and counts do not.
- **Screenshots:** the terminal-style images in Output_Screenshots are rendered from the captured console output of the scripts (so that they match the files in results/). The graphs are produced directly by matplotlib in the scripts.

Q1 – Sorting Under Different Data Conditions (6 marks)

1.1 Problem and measures

A fulfilment centre must order customer orders by dispatch priority score. The three candidates are Selection, Bubble and Insertion Sort. The question is whether one algorithm should always be preferred or whether the choice depends on the input arrangement. Measures: key comparisons and data movements (swaps for Selection/Bubble; element shifts for Insertion), plus execution time as supporting evidence.

1.2 Algorithm design and pseudocode

```
SelectionSort(A[0..n-1])
  for i <- 0 to n-2
      minIdx <- i
      for j <- i+1 to n-1
          comparisons++
          if A[j] < A[minIdx] then minIdx <- j
      if minIdx != i then
          swap A[i], A[minIdx] ; movements++


BubbleSort(A[0..n-1])
  for pass <- 0 to n-2
      swapped <- false
      for j <- 0 to n-2-pass
          comparisons++
          if A[j] > A[j+1] then swap A[j], A[j+1] ; movements++ ; swapped <- true
      if swapped = false then exit      // already sorted


InsertionSort(A[0..n-1])
  for i <- 1 to n-1
      key <- A[i] ; j <- i-1
      while j >= 0
          comparisons++
          if A[j] > key then A[j+1] <- A[j] ; movements++ ; j <- j-1
          else break
      A[j+1] <- key
```

Bubble Sort includes the usual early-exit flag (stop when a full pass makes no swap); without it Bubble Sort could not benefit from sorted data at all. Insertion Sort stops its inner loop at the first element not greater than the key.

1.3 Prediction before implementation (with algorithmic reasons)

- Selection: comparisons = n(n-1)/2 = 499,500 on EVERY dataset (no early exit); swaps at most n-1 (about n-1 for random, about n/2 for reverse).

- Bubble: swaps = I; comparisons close to n(n-1)/2 for D1 and D3 (about 495k for random, exactly 499,500 for reverse). For D2 I expect it to stay expensive (several hundred thousand comparisons) because each late small element crawls left one position per pass, so about as many passes as the largest displacement are needed.
- Insertion: comparisons about I + n - 1, shifts = I. D1: I about 249,750 -> about 250k. D2: I about 50 x 475 + 50x49/4 about 24.4k -> about 25k comparisons (around 20 times fewer than Selection). D3: I = 499,500 -> 499,500 comparisons and shifts.
- Sorted input: Bubble and Insertion about n-1 = 999 comparisons, 0 movements; Selection still 499,500.
- Recommendation hypothesis: Insertion Sort for the 95%-sorted workload.

Reasons. Selection Sort always scans the entire unsorted suffix to find the minimum, so its comparison count is a fixed n(n−1)/2 and never depends on the data. Bubble Sort and Insertion Sort remove one inversion per swap/shift, so their movements equal the number of inversions I; Insertion Sort stops scanning as soon as the correct place is found, so its comparisons are about I + n − 1. Bubble Sort, however, moves a small element left by only one position per pass, so a few badly placed late elements force many passes.

1.4 Designed test datasets (n = 1000, fixed seed)

Dataset	Construction	Why it was chosen
D1 Random (contrast A)	random permutation of priority scores	typical average case; I ≈ $n^2/4$
D2 Nearly sorted (contrast B)	first 950 scores sorted, newest 50 (5%) random scores appended at the end	exactly the reported scenario: 95% already in order, periodic small batches
D3 Reverse sorted (my edge case)	scores in strictly descending order	maximum number of inversions n(n−1)/2; exposes worst case of every algorithm
Extra: already sorted / all equal	sorted list; every score equal	sanity checks for early exit and duplicates (stability: strict comparisons leave equal keys untouched)

1.5 Results

Table 1.1 – Operation counts, n = 1000 (raw data: results/q1_raw.csv)

Dataset	Algorithm	Inversions	Comparisons	Movements	Time (ms)
D1 Random	Selection	238,575	499,500	995	24.665
D1 Random	Bubble	238,575	498,680	238,575	46.966
D1 Random	Insertion	238,575	239,569	238,575	22.823
D2 Nearly sorted (95%)	Selection	22,598	499,500	972	24.28
D2 Nearly sorted (95%)	Bubble	22,598	498,680	22,598	30.864
D2 Nearly sorted (95%)	Insertion	22,598	23,597	22,598	2.229
D3 Reverse sorted	Selection	499,500	499,500	500	24.234
D3 Reverse sorted	Bubble	499,500	499,500	499,500	59.877
D3 Reverse sorted	Insertion	499,500	499,500	499,500	45.841
Extra: Already sorted	Selection	0	499,500	0	23.803
Extra: Already sorted	Bubble	0	999	0	0.059
Extra: Already sorted	Insertion	0	999	0	0.088
Extra: All equal	Selection	0	499,500	0	23.19
Extra: All equal	Bubble	0	999	0	0.057
Extra: All equal	Insertion	0	999	0	0.085

Table 1.2 – Scaling on D2 (5% new batch, n = 250 … 2000)

n	Algorithm	Comparisons	Movements	Time (ms)
250	Selection	31,125	245	1.323
250	Bubble	31,115	1,457	1.601
250	Insertion	1,706	1,457	0.145
500	Selection	124,750	486	5.878
500	Bubble	124,705	6,686	6.573
500	Insertion	7,185	6,686	0.639
1000	Selection	499,500	972	54.581
1000	Bubble	498,680	22,598	35.283
1000	Insertion	23,597	22,598	2.303
2000	Selection	1,999,000	1,973	97.991
2000	Bubble	1,998,010	88,169	126.576
2000	Insertion	90,168	88,169	9.135

Table 1.3 – Sensitivity: comparisons as the appended batch grows (n = 1000)

New batch %	Selection	Bubble	Insertion
0	499,500	999	999
1	499,500	498,419	5,263
5	499,500	498,834	26,288
10	499,500	498,834	50,730
25	499,500	498,834	109,890
50	499,500	498,834	194,088
100	499,500	498,834	255,011

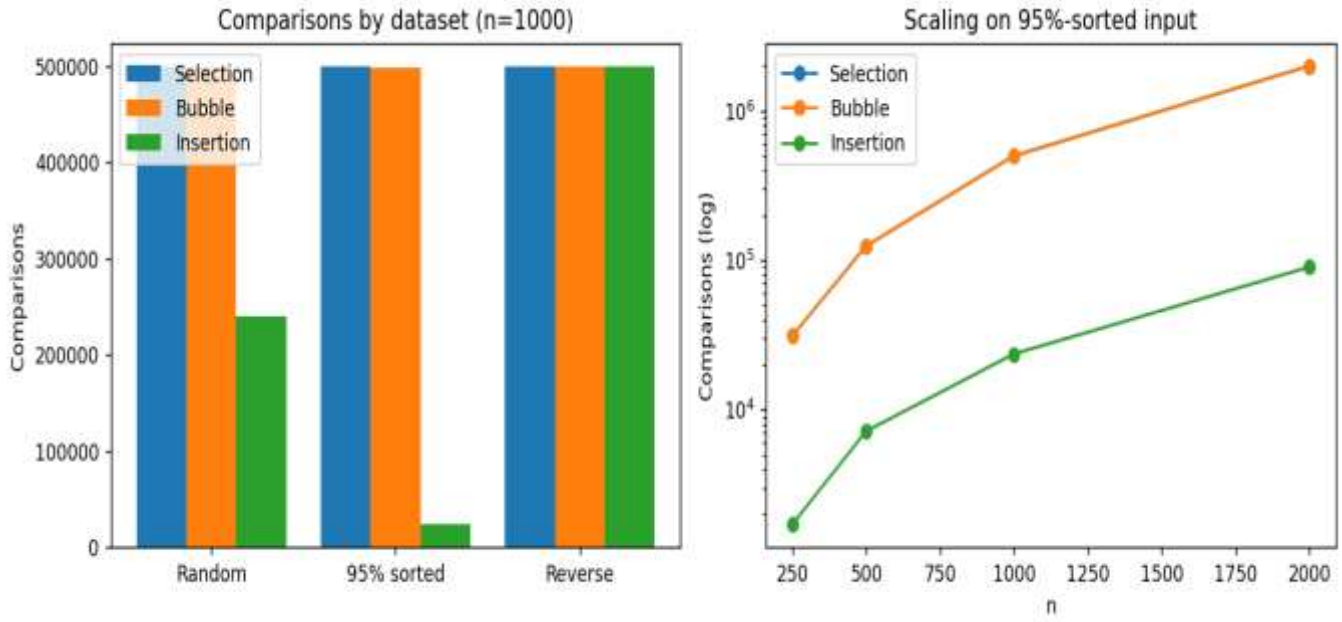


Figure 1.1 – Left: comparisons per dataset. Right: comparisons on the 95%-sorted workload as n grows (log scale).

q1_sorting.py (part 1/2)

```
$ python q1_sorting.py
=== Q1: sorting comparison, n = 1000 ===
Dataset                  Algorithm   Inversions  Comparisons  Movements  Time(ms)
D1 Random                Selection       238575       499500        995    24.665
D1 Random                Bubble          238575       498680     238575    46.966
D1 Random                Insertion       238575       239569     238575    22.823
D2 Nearly sorted (95%)   Selection        22598       499500        972     24.28
D2 Nearly sorted (95%)   Bubble           22598       498680      22598    30.864
D2 Nearly sorted (95%)   Insertion        22598        23597      22598     2.229
D3 Reverse sorted        Selection       499500       499500        500    24.234
D3 Reverse sorted        Bubble          499500       499500     499500    59.877
D3 Reverse sorted        Insertion       499500       499500     499500    45.841
Extra: Already sorted    Selection            0       499500          0    23.803
Extra: Already sorted    Bubble               0          999          0     0.059
Extra: Already sorted    Insertion            0          999          0     0.088
Extra: All equal         Selection            0       499500          0     23.19
Extra: All equal         Bubble               0          999          0     0.057
Extra: All equal         Insertion            0          999          0     0.085

Theory reference (n=1000): n(n-1)/2 = 499500

=== Scaling on D2 (95% already sorted, 5% new batch appended) ===
     n  Algorithm  Comparisons  Movements  Time(ms)
   250  Selection        31125        245     1.323
   250     Bubble        31115       1457     1.601
   250  Insertion         1706       1457     0.145
   500  Selection       124750        486     5.878
   500     Bubble       124705       6686     6.573
   500  Insertion         7185       6686     0.639
  1000  Selection       499500        972    54.581
  1000     Bubble       498680      22598    35.283
  1000  Insertion        23597      22598     2.303
  2000  Selection      1999000       1973    97.991
  2000     Bubble      1998010      88169   126.576
  2000  Insertion        90168      88169     9.135

=== Sensitivity: size of the new batch (n=1000) ===
 new %  Selection   Bubble  Insertion   (comparisons)
     0     499500      999        999
     1     499500   498419       5263
     5     499500   498834      26288
```

Figure 1.2 – Execution screenshot (part 1)

q1_sorting.py (part 2/2)

```
$ python q1_sorting.py
    10     499500   498834      50730
    25     499500   498834     109890
    50     499500   498834     194088
   100     499500   498834     255011
```

Figure 1.3 – Execution screenshot (part 2)

1.6 Prediction versus observation

Prediction	Observed	Verdict
Selection: 499,500 comparisons on every dataset	499,500 on all five datasets	Confirmed exactly
Selection swaps ≤ n−1; about n−1 for random, about n/2 for reverse	995 (random), 972 (D2), 500 (reverse)	Confirmed
Bubble/Insertion movements = number of inversions	movements = inversions in every row	Confirmed exactly
Insertion comparisons ≈ I + n − 1	D2: 22,598 + 999 = 23,597 (exact); D1: 239,569	Confirmed (differs by ≤ 5 where the key reaches index 0)
D1 inversions ≈ 249,750; D2 ≈ 24.4k	238,575 and 22,598 (about 4–7% lower)	Right order of magnitude; my expected values were averages and a single permutation deviates
Bubble stays expensive on D2 (several hundred thousand)	498,680 comparisons – the same as for random data	Confirmed, and stronger than expected: the cost barely depends on the size of the new batch (1% new → 498,419)
Reverse: all three ≈ 499,500 comparisons	499,500 for all three; Bubble and Insertion move 499,500 elements	Confirmed
Sorted input: Bubble and Insertion n−1 = 999	999 and 0 movements (Selection still 499,500)	Confirmed

1.7 Explanation of the behaviour

- **Selection Sort** is data-insensitive: it is the only algorithm whose comparison count is identical on sorted, random and reversed input. Its one strength is few movements (at most n−1 swaps; 972 on D2).
- **Bubble Sort** with early exit helps only if the data are already completely sorted. On D2 the newest orders (appended at the end) have small priority scores that must travel far to the left, and an element moves only one position per pass. So about n passes are needed and the comparison count stays near n(n−1)/2 – which is why D2 behaves like random data for Bubble Sort.
- **Insertion Sort** performs work proportional to n + I. On D2 only the 50 new orders create inversions (22,598 in total), so it needs 23,597 comparisons – **21.2 times fewer** than Selection and **21.1 times fewer** than Bubble – and 2.2 ms against 24.3 ms and 30.9 ms.

- Because the new batch grows with n in my scaling run (5% of n), Insertion Sort remains $\Theta(n^2)$ there, but with a constant of about $0.023n^2$ compared with $0.5n^2$ (Table 1.2: 90,168 vs 1,999,000 at n = 2000). If the batch size were a fixed number k the cost would be $\Theta(kn)$.

1.8 Design decision

Recommendation: Insertion Sort, defended with evidence: (i) 23,597 comparisons against ≈ 499,000 for the other two on the 95%-sorted workload; (ii) its cost tracks the amount of disorder (it is adaptive), and Table 1.3 shows it is never worse in comparisons at any batch size from 0% to 100% (tied with Bubble Sort only when there is no new batch) and far better for small batches; (iii) it is stable and sorts in place; (iv) it handles the 'periodic small batches' naturally because each new order is inserted into an already ordered sequence. Selection Sort should be preferred only where moving a record is extremely expensive and comparisons are cheap (it needs at most n−1 swaps, whereas Insertion Sort here performed 22,598 shifts). Bubble Sort is never the best choice in the experiments. Caveat: when a batch becomes large relative to the list, none of these three is the right tool (a faster general algorithm would be needed), but that lies beyond the brute-force scope of CO2.

Q2 – When Is Sequential Search an Appropriate Engineering Choice? (4 marks)

2.1 Design of the experiments

```
SequentialSearch(L[0..n-1], key)
  comparisons <- 0
  for i <- 0 to n-1
      comparisons++
      if L[i].ToolID = key then return (i, L[i], comparisons)    // successful
  return (NOT_FOUND, comparisons)                                  // unsuccessful: n
comparisons
```

- Successful searches at first / 25% / middle / 75% / last position, for list sizes n = 100, 1,000, 10,000 and 100,000.
- Unsuccessful search (tool already returned) for the same sizes.
- Average over 1,000 random successful lookups per size.
- Measured cost per comparison and a request-rate model to convert comparisons into latency and CPU load.
- Operating boundary for a latency budget (50 ms) and a CPU budget (10% of one core); reference design: hash index on Tool ID.

2.2 Predictions (written before execution)

- Successful at position p (1-based) costs exactly p comparisons; first = 1, last = n, middle about n/2.
- Unsuccessful always costs exactly n comparisons (worst case).
- Mean successful cost = (n+1)/2.
- Pure-Python cost per comparison about 50-150 ns, so a full scan of 100,000 records is about 5-15 ms.
- One hangar (hundreds of records, a few requests per minute): sequential search is fine. For 20 facilities the per-request time stays small in isolation, but the total scan work grows with (records x request rate); I expect the boundary to be driven by the request RATE (hundreds of requests per second) rather than the dataset size alone, and I expect an indexed structure to become the better choice well before then.

2.3 Comparison-count evidence

Table 2.1 – Comparisons for successful and unsuccessful searches

n	first	25%	middle	75%	last	unsuccessful	avg successful (1000 random)	(n+1)/2
100	1	25	50	75	100	100	49.5	50.5
1,000	1	250	500	750	1,000	1,000	487.1	500.5
10,000	1	2,500	5,000	7,500	10,000	10,000	4847.9	5000.5
100,000	1	25,000	50,000	75,000	100,000	100,000	48531.2	50000.5

The counts match the prediction exactly: a successful search at position p costs p comparisons; an unsuccessful search always costs n comparisons (the worst case, because absence can only be proven by examining every record); the mean for random successful lookups is within 4% of (n+1)/2 (the small deviation is sampling noise of 1,000 draws).

2.4 From comparisons to an operating boundary

The measured cost was **61.8 ns per comparison** (best of 7 full scans of 100,000 records; this includes Python loop overhead, so it is a conservative figure for this environment). A dictionary lookup on Tool ID cost 55.6 ns including the same loop overhead, i.e. it does not depend on n.

Table 2.2 – Latency of one sequential search

n	avg successful (n/2)	unsuccessful (n)
100	0.0031 ms	0.0062 ms
1,000	0.0309 ms	0.0618 ms
10,000	0.3088 ms	0.6176 ms
100,000	3.0880 ms	6.1761 ms
1,000,000	30.8803 ms	61.7607 ms

Table 2.3 – Operating boundary. Latency boundary for a 50 ms budget: n ≤ 809,576 records, independent of request rate

requests / second	max n for 10% CPU (average scan n/2)	max n for 10% CPU (unsuccessful, n)
0.1	32,383,063	16,191,531
1	3,238,306	1,619,153
10	323,830	161,915
100	32,383	16,191
1000	3,238	1,619
10000	323	161

Boundary rule derived from the evidence: the load on the processor is f × (n/2) × t_c (average successful scan) so the CPU budget is respected while **f × n ≤ 2 × 0.10 / t_c ≈ 3.2 × 10^6 record-comparisons per second**, and a single request stays under 50 ms while n ≤ ~810,000. Beyond either limit, sequential search should no longer be recommended.

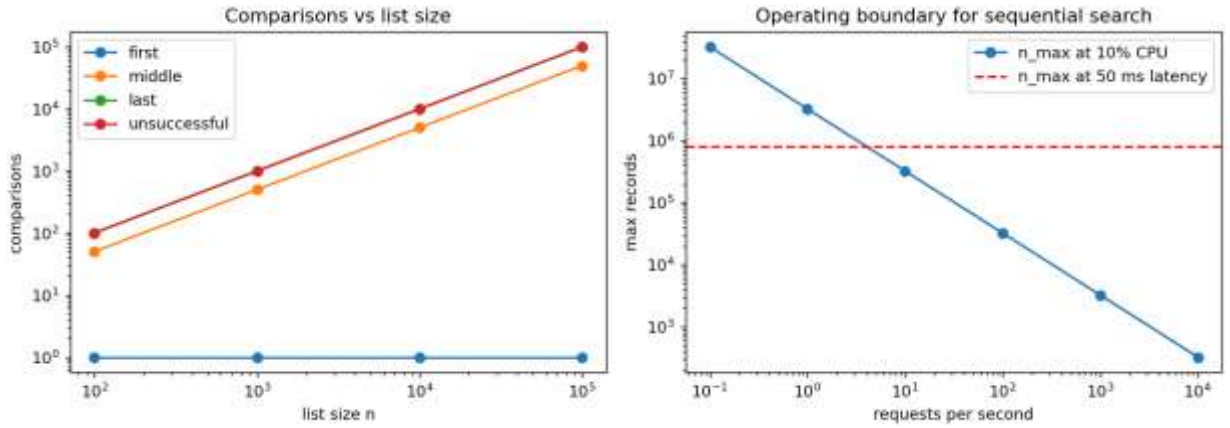


Figure 2.1 – Left: comparisons against list size. Right: largest list that sequential search can serve within the 10% CPU budget as the request rate grows (red line: 50 ms latency limit).

```
q2_sequential_search.py  (part 1/1)

$ python q2_sequential_search.py
=== Q2 Part 1: comparisons for successful / unsuccessful searches ===
       n   first     25%  middle     75%    last  unsucc.  avg succ (1000 rnd)   (n+1)/2
     100       1      25      50      75     100      100                 49.5      50.5
    1000       1     250     500     750    1000     1000                487.1     500.5
   10000       1    2500    5000    7500   10000    10000               4847.9    5000.5
  100000       1   25000   50000   75000  100000   100000              48531.2   50000.5

Measured cost per comparison (best of 7 full scans of 100,000): 61.8 ns
Measured hash-index lookup (dict on ToolID): 55.6 ns per lookup

=== Q2 Part 2: latency of one sequential search (avg successful = n/2, unsuccessful = n) ===
       n   avg successful   unsuccessful
     100        0.0031 ms      0.0062 ms
    1000        0.0309 ms      0.0618 ms
   10000        0.3088 ms      0.6176 ms
  100000        3.0880 ms      6.1761 ms
 1000000       30.8803 ms     61.7607 ms

=== Q2 Part 3: operating boundary (latency budget 50 ms, CPU budget 10% of one core) ===
Latency boundary: worst-case scan <= 50 ms -> n <= 809,576 records (independent of request rate)
  requests/s     max n (CPU 10%, avg n/2 scan)    max n (unsuccessful, n scan)
         0.1                      32,383,063                      16,191,531
           1                       3,238,306                       1,619,153
          10                         323,830                         161,915
         100                          32,383                          16,191
        1000                           3,238                           1,619
       10000                             323                             161

=== Q2 Part 4: changed condition - one hangar vs 20 facilities (model) ===
scenario                                        records   req/s  scan ms (n/2)  CPU load %  hash CPU %
1 hangar                                            800     0.2         0.0247      0.0005    0.000001
20 facilities (x20 records, x20 requests)        16,000     4.0         0.4941      0.1976    0.000022
20 facilities, busy shift peak (x10 burst)       16,000    40.0         0.4941      1.9763    0.000222
20 facilities, organisation-wide dashboards/bots 16,000   500.0         0.4941     24.7043    0.002781
```

Figure 2.2 – Execution screenshot

2.5 Changed condition: one hangar → 20 facilities

Assumed scenario values (my own modelling assumptions, not data supplied with the assignment): one hangar ≈ 800 active records and 0.2 lookups/s; twenty facilities ≈ 16,000 records and 4 lookups/s on average, bursts ten times higher, and an organisation-wide dashboard/bot load of 500 lookups/s.

Table 2.4 – Re-evaluation under the changed condition

Scenario	records	req/s	avg scan (ms)	CPU load of sequential search (%)	CPU load of hash index (%)
1 hangar	800	0.2	0.025	0.000	0.000001
20 facilities (x20 records, x20 requests)	16,000	4.0	0.494	0.198	0.000022
20 facilities, busy shift peak (x10 burst)	16,000	40.0	0.494	1.976	0.000222
20 facilities, organisation-wide dashboards/bots	16,000	500.0	0.494	24.704	0.002781

Re-evaluated recommendation. For one hangar, sequential search is entirely reasonable (0.025 ms per lookup, simple, no index to maintain while records change continuously). For 20 facilities at the assumed average and burst loads it is still within budget (0.2% and 2% CPU, 0.49 ms per lookup), so the data do not force an immediate change. The boundary is crossed when the combined load reaches $f \times n \approx 3.2 \times 10^6$: with 16,000 records that is about 200 requests/s, and my organisation-wide dashboard scenario (500 req/s, 24.7% CPU) is beyond it. I therefore recommend: keep sequential search for the per-hangar local list, but for the consolidated 20-facility service use an index keyed by Tool ID (hash table), because lookups become independent of the number of records, issue/return updates are O(1), and the system has headroom for growth. The decision threshold is the measured rule above, not intuition.

Q3 – Engineering the Behaviour of Brute-Force String Matching (6 marks)

3.1 Algorithm

```
BruteForceMatch(T[0..n-1], P[0..m-1])
  comparisons <- 0
  for i <- 0 to n-m                    // every alignment of P under T
      j <- 0
      while j < m
          comparisons++                // one character comparison
          if T[i+j] != P[j] then break
          j <- j+1
      if j = m then return i, comparisons   // first match
  return NOT_FOUND, comparisons
```

3.2 Test conditions and predictions (written before execution)

- **A (low work):** T = "abcdefghij" repeated to n = 10,000; P = "ZERO#ALRT" (m = 9, first character never occurs in T).
- **B (engineered high work):** T = "A" x 10,000; P = "A" x 49 + "B" (m = 50).
- **C (own choice, realistic log):** synthetic authentication log lines, with the target signature "AUTH FAILED user=root" occurring only in the very last line, while all earlier lines share the long prefix "AUTH FAILED user=" (partial matches).
- Extras: D = pattern found at the very start; E = scaling of A and B for n = 1,000 ... 32,000; F = effect of m on B.

Predictions and the algorithmic reason for each:

- A: no alignment can pass the first character, so comparisons = n-m+1 = 9,992.
- B: each of the n-m+1 = 9,951 alignments matches 49 characters and fails on the 50th, so comparisons = 9,951 x 50 = 497,550 (about 50 times more than A for the same n).
- C: the long common prefix makes many alignments run about 17 characters before failing; I expect the count to be somewhere between 1.1n and 1.5n (clearly above A, far below B).
- D: m comparisons only.
- Same n, same algorithm, work differs by ~50x, so input size n alone cannot describe the cost; the repeated-prefix structure of text and pattern does.

3.3 Results

Table 3.1 – Character comparisons for the designed conditions

Test	n	m	match at	comparisons	comparisons / n	time (ms)
A low work	10,000	9	-1	9,992	0.999	0.981
B high work	10,000	50	-1	497,550	49.755	40.567
C log text	10,163	21	10128	13,389	1.317	1.305
D match at start	10,021	21	0	21	0.002	0.002
E pattern longer than text	3	6	-1	0	0.0	0.0

Table 3.2 – Scaling with text length n (pattern length fixed)

n	A: comparisons	A: n−m+1	B (m=50): comparisons	B: (n−m+1)·m	B ÷ A
1,000	992	992	47,550	47,550	47.9
2,000	1,992	1,992	97,550	97,550	49.0
4,000	3,992	3,992	197,550	197,550	49.5
8,000	7,992	7,992	397,550	397,550	49.7
16,000	15,992	15,992	797,550	797,550	49.9
32,000	31,992	31,992	1,597,550	1,597,550	49.9

Table 3.3 – Effect of pattern length m on worst-case text (n = 10,000)

m	comparisons	(n−m+1)·m
5	49,980	49,980
10	99,910	99,910
25	249,400	249,400
50	497,550	497,550
100	990,100	990,100
200	1,960,200	1,960,200
500	4,750,500	4,750,500
1,000	9,001,000	9,001,000
2,500	18,752,500	18,752,500
5,000	25,005,000	25,005,000

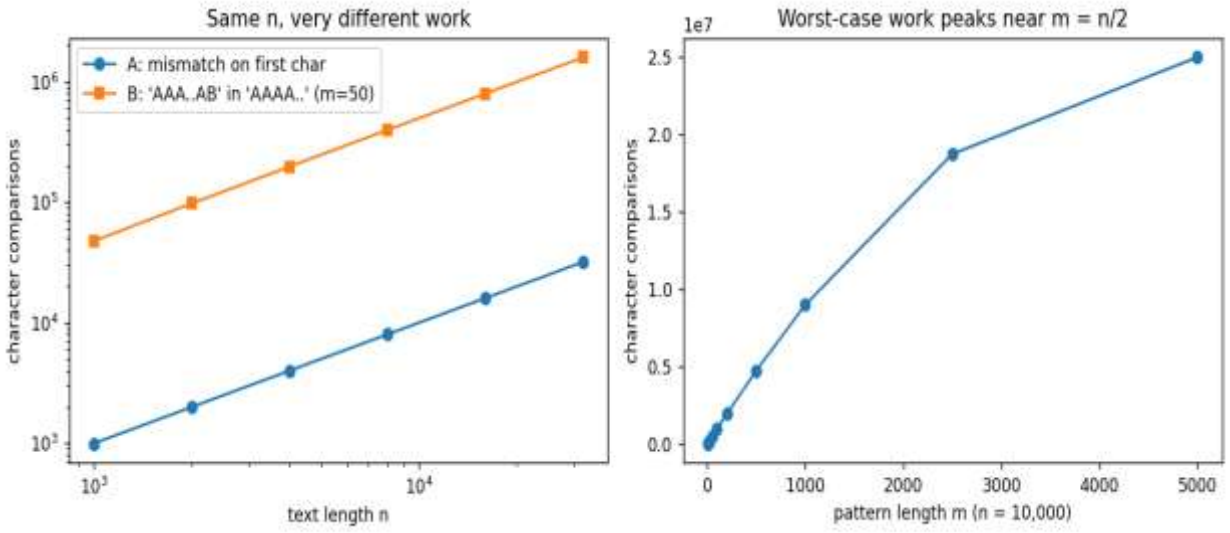


Figure 3.1 – Left: identical n, very different work. Right: effect of m on the engineered worst case.

q3_string_matching.py (part 1/1)

```
$ python q3_string_matching.py
=== Q3: brute-force string matching - three designed conditions ===
A low work : n=10000, m=9, match at -1, comparisons=9,992, comps/n=1.00, time=0.98 ms
    prediction: n-m+1 = 9,992
B high work: n=10000, m=50, match at -1, comparisons=497,550, comps/n=49.76, time=40.57 ms
    prediction: (n-m+1)*m = 497,550
C log text : n=10163, m=21, match at 10128, comparisons=13,389, comps/n=1.32, time=1.30 ms
    prediction: 11,179 .. 15,244

=== Extra tests ===
D match at start: n=10021, m=21, match at 0, comparisons=21, comps/n=0.00, time=0.00 ms
    prediction: m = 21
E pattern longer than text: n=3, m=6, match at -1, comparisons=0, comps/n=0.00, time=0.00 ms
    prediction: 0 (no alignment)

=== Scaling with n (m fixed) ===
       n    A: comps    A: n-m+1  B(m=50): comps    B: (n-m+1)m  ratio B/A
    1000         992         992          47,550        47,550       47.9
    2000       1,992       1,992          97,550        97,550       49.0
    4000       3,992       3,992         197,550       197,550       49.5
    8000       7,992       7,992         397,550       397,550       49.7
   16000      15,992      15,992         797,550       797,550       49.9
   32000      31,992      31,992       1,597,550     1,597,550       49.9

=== Effect of pattern length m on text of n=10,000 'A's (B-type pattern) ===
     m   comparisons     (n-m+1)*m
     5        49,980        49,980
    10        99,910        99,910
    25       249,400       249,400
    50       497,550       497,550
   100       990,100       990,100
   200     1,960,200     1,960,200
   500     4,750,500     4,750,500
  1000     9,001,000     9,001,000
  2500    18,752,500    18,752,500
  5000    25,005,000    25,005,000
```

Figure 3.2 – Execution screenshot

3.4 Prediction versus observation and explanation

Test	Prediction	Observed	Verdict
A	9,992	9,992	exact
B	497,550	497,550	exact
C (log text)	between 11,179 and 15,244 (1.1n–1.5n)	13,389 (1.32n)	inside the predicted range
D (match at start)	m = 21	21	exact
E (pattern longer than text)	0	0	exact

- **Why B needs ~50 times more work than A** although n is the same: in A every alignment fails on the very first character (the pattern's first letter never occurs in the text), costing one comparison per alignment. In B every alignment matches 49 characters before failing on the last one, costing m = 50 comparisons per alignment. After each failure the brute-force method forgets everything it learned and starts the next alignment from the beginning of the pattern.
- **Characteristics that create repeated comparisons:** (1) a long common prefix between the pattern and many text positions (test C: all log lines begin with 'AUTH FAILED user=', so each line costs about 17 comparisons before the real difference appears); (2) a small alphabet / highly repetitive text (test B), so that partial matches are everywhere; (3) the mismatch occurring late in the pattern; (4) a long pattern (Table 3.3: cost = $(n-m+1)\cdot m$ grows with m, peaking near $m \approx n/2$).
- **What this says about measuring algorithms by input size alone:** the three texts A, B and C all have $n \approx 10^4$, yet the costs are 9,992, 13,389 and 497,550. The 'O(nm)' bound is attained only for special structure, the best case is $\Theta(n)$, and a realistic log (C) sits close to the best case. Input size alone predicts neither; the structure of the text/pattern pair (how long partial matches are) must be stated – this is why best/average/worst cases are analysed separately and why empirical counting on representative data matters.

Q4 – Closest Pair and the Cost of Examining Every Possibility (6 marks)

4.1 Design

```
ClosestPair(D[0..n-1])        // D[i] = (x_i, y_i)
  best <- infinity ; pairsExamined <- 0 ; distCalcs <- 0
  for i <- 0 to n-2
      for j <- i+1 to n-1                 // j > i : each unordered pair once
          pairsExamined++
          d2 <- (xi-xj)^2 + (yi-yj)^2     // squared distance, no sqrt needed to compare
          distCalcs++
          if d2 < best then best <- d2 ; (a,b) <- (i,j)
  return D[a], D[b], sqrt(best)
```

Which pairs must be examined? Every unordered pair of distinct drones, because without further information any two drones could be the closest. **Avoiding duplicates:** fix j > i: pair (i, j) and (j, i) have the same distance, and i = j is a drone compared with itself. **Squared distances** are compared, so no square root is needed in the inner loop (the root is taken once at the end).

4.2 Mathematical derivation of the number of candidate pairs

```
Outer index i = 0 .. n-2 has (n-1-i) partners j > i.
Total = sum_{i=0}^{n-2} (n-1-i) = (n-1) + (n-2) + ... + 2 + 1
      = (n-1)n / 2          (arithmetic series: count x (first+last)/2)
Check by choosing 2 drones out of n:  C(n,2) = n! / (2!(n-2)!) = n(n-1)/2
Example: n = 4 -> 6 pairs (verified by the demo run); n = 1000 -> 499,500.
Growth: n(n-1)/2 = n^2/2 - n/2  ->  Theta(n^2); doubling n multiplies the pairs by ~4.
```

4.3 Predictions (written before execution)

- Pairs examined = distance calculations = n(n-1)/2 exactly: 4,950; 124,750; 499,500; 1,999,000; 7,998,000; 31,996,000.
- Doubling n multiplies the time by about 4.
- Pure Python about 0.2-0.5 microseconds per pair, so n = 2,000 about 0.4-1 s: boundary of practicality expected around n = 2,000-3,500.
- The inefficiency comes from comparing pairs that are obviously far apart; a redesign must avoid looking at far-apart pairs (e.g. by spatial ordering/partitioning), not just compute a distance faster.

4.4 Experimental results

Table 4.1 – Brute-force closest pair (7 input sizes; raw data results/q4_raw.csv)

n	pairs examined	n(n−1)/2	distance calcs	time (s)	µs per pair	closest distance
100	4,950	4,950	4,950	0.0008	0.1521	15.0
250	31,125	31,125	31,125	0.0047	0.1494	15.0
500	124,750	124,750	124,750	0.0195	0.1566	15.0
1,000	499,500	499,500	499,500	0.0795	0.1591	1.0
2,000	1,999,000	1,999,000	1,999,000	0.3389	0.1695	1.0
4,000	7,998,000	7,998,000	7,998,000	1.4491	0.1812	1.0
8,000	31,996,000	31,996,000	31,996,000	6.6259	0.2071	0.0

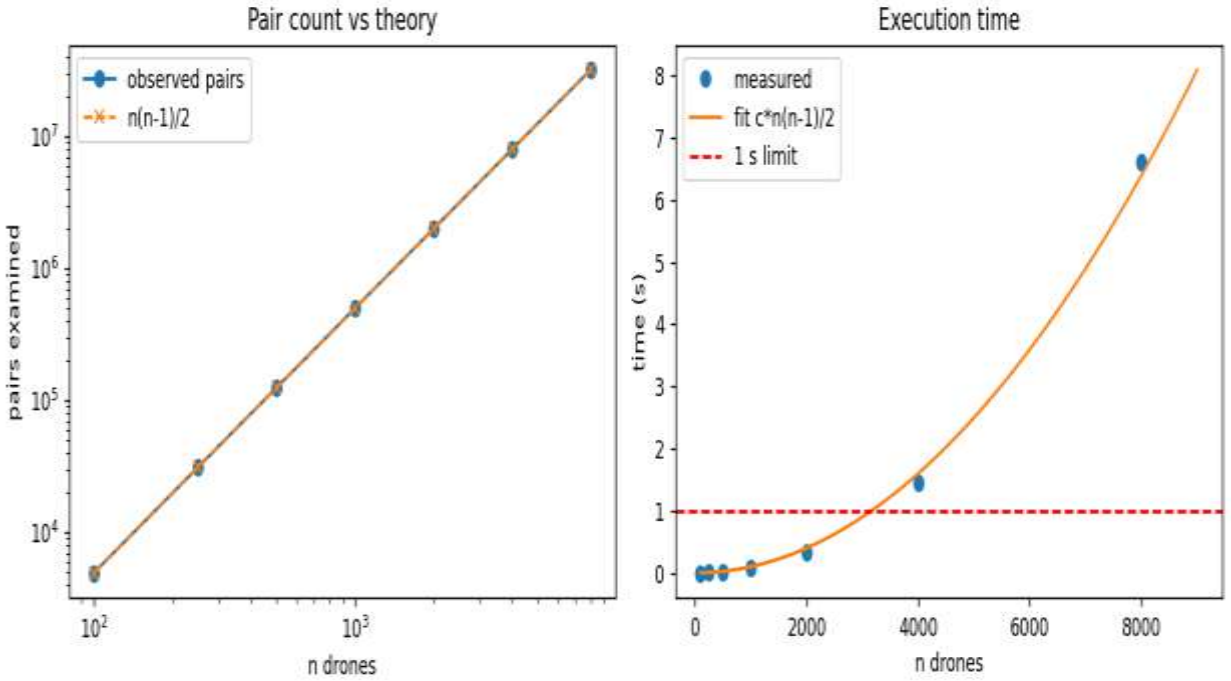


Figure 4.1 – Left: observed pair count coincides with n(n−1)/2. Right: measured time against the fitted quadratic and the 1-second limit.

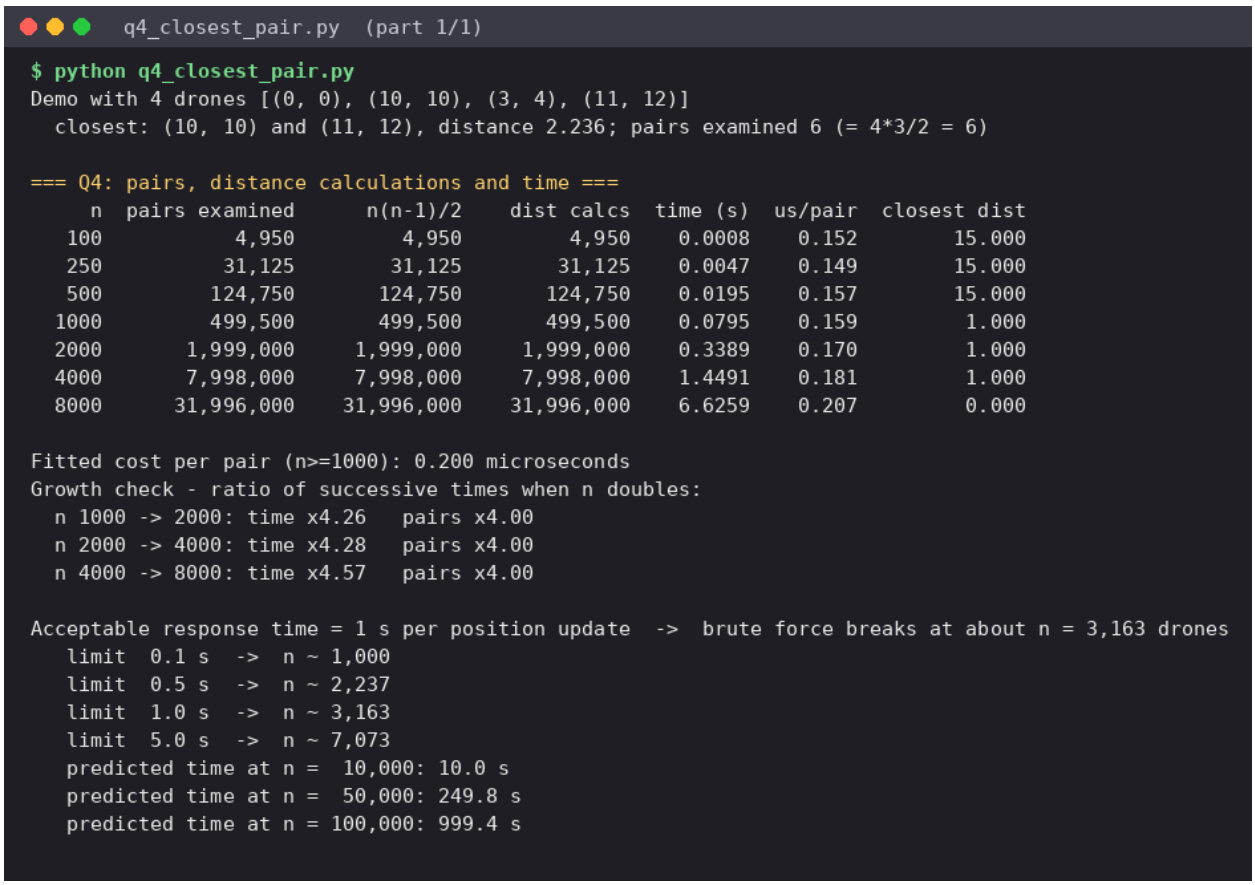


Figure 4.2 – Execution screenshot

Theory versus experiment. The number of pairs and distance calculations equals n(n−1)/2 exactly for all seven sizes (assertion in the program). Time grows by a factor of 4.26–4.57 each time n doubles (the pair count grows by exactly 4); the slightly larger factor is explained by the cost per pair rising from 0.15 µs to 0.21 µs as the list of 8,000 tuples no longer fits the processor caches as comfortably. A least-squares-style fit gives c = 0.200 µs per pair. At n = 8,000 two drones were reported at distance 0 (identical coordinates on the integer grid) – a genuine collision-risk finding that the algorithm handles correctly.

Prediction check: pairs – exact. Doubling ⇒ ≈ 4× time – confirmed (4.3–4.6×). My predicted cost of 0.2–0.5 µs per pair was slightly pessimistic at small n (0.15–0.18 µs measured); the boundary I predicted (2,000–3,500 drones) is consistent with the measurement below.

4.5 Scalability inquiry

Acceptability criterion (defined by me): one position update must be fully checked within **1 second**, because drones report about once per second and a slower check cannot keep up with the data stream. With c = 0.200 µs/pair, $c \cdot n(n-1)/2 = 1$ s gives **n ≈ 3,163 drones**; the measured value at n = 4,000 was already 1.45 s while 2,000 drones took 0.34 s. Other limits: 0.1 s → n ≈ 1,000; 0.5 s → 2,237; 5 s → 7,073. Predicted time for 10,000 drones ≈ 10 s, for 100,000 drones ≈ 1,000 s (extrapolated from the quadratic fit).

Where the inefficiency originates. All the work is in examining pairs: $\Theta(n^2)$ pair examinations, each costing a constant amount, so the cost of a single pair is not the problem – the number of pairs is. The method is also blind to geometry: it examines pairs of drones that are kilometres apart although such a pair can never be the closest. It repeats the same kind of work for every drone and uses nothing learned from earlier comparisons. **A redesigned strategy would need to** avoid examining pairs that are provably too far apart (for example by exploiting an ordering or spatial partitioning of the positions so that each drone is only compared with nearby drones), and to reuse the current best distance as a cut-off radius – so that the number of candidate pairs examined grows much more slowly than $n^2/2$. (Not implemented here, as the question does not require it.)

Q5 – Convex Hull Derived from a Geometric Principle (7 marks)

5.1 From the geometric principle to a decision rule (student-derived)

- **Two points define a line.** For p1 = (x1, y1) and p2 = (x2, y2): a·x + b·y = c with a = y2 − y1, b = x1 − x2, c = x1·y2 − y1·x2.
- **Position of another point q:** evaluate s(q) = a·qx + b·qy − c. It is zero on the line, positive on one side and negative on the other (it equals the cross product (p2 − p1) × (q − p1)). With integer coordinates the test is exact.
- **How the remaining points are examined:** every point k ≠ i, j is evaluated; the loop stops early as soon as both a positive and a negative sign have been seen (the line then cannot be a boundary line).
- **Condition identifying a potential hull edge:** all s(q) ≥ 0 (or all ≤ 0). Points with s = 0 are collinear with the line; they are allowed only inside the segment p1p2 – otherwise p1p2 is only part of a longer edge whose true end points are another pair.
- **Constructing the boundary:** a pair accepted with all points on the left becomes a directed edge i → j (all on the right: j → i). The edges are stored in a successor map; starting at the lowest-leftmost point and following the map returns the vertices in counter-clockwise order. Duplicate points are removed first.

5.2 Pseudocode

```
ConvexHull(P)
  P <- remove duplicate points
  if |P| < 3 or all points collinear: return the two extreme points (or the points)
  edges <- empty map ; tests <- 0
  for each unordered pair (i, j), i < j
      seenPos <- false ; seenNeg <- false ; ok <- true
      a,b,c <- line through P[i], P[j]
      for each k != i, j
          tests++
          s <- a*P[k].x + b*P[k].y - c
          if s > 0 then seenPos <- true
          if s < 0 then seenNeg <- true
          if s = 0 and P[k] is outside segment P[i]P[j] then ok <- false ; break
          if seenPos and seenNeg then ok <- false ; break        // early exit
      if ok then
          if not seenNeg then edges[i] <- j        // all on left: directed i -> j
          else                edges[j] <- i        // all on right: directed j -> i
  start <- lowest then leftmost point ; walk edges from start until returning
```

5.3 Predictions (written before execution) and test configurations

- T1 hull = the visually extreme points; interior points never appear.
- T2 hull = only the 4 corners (collinear mid-side points rejected by the segment rule, duplicates removed).
- T3 returns the 2 extreme points only.
- T4 returns all n points.
- Work without early exit is exactly n(n-1)(n-2)/2 (exponent 3). With early exit most non-hull pairs are rejected after a few points, so random-point work should look closer to n^2 (exponent about 2-2.5).

5.4 Results and visualisation

Table 5.1 – Hull of each test configuration

Configuration	Hull vertices found
T1 mixed boundary/interior	6: (8,1), (2,3), (1,11), (9,15), (15,14), (18,6)
T2 designed: corners + on-edge + duplicates + interior	4: (0,0), (0,10), (10,10), (10,0)
T3 all collinear	2: (0,1), (7,15)
T4 12 points on a circle	12: (0,-100), (-50,-87), (-87,-50), (-100,0), (-87,50), (-50,87), (0,100), (50,87), (87,50), (100,0), (87,-50), (50,-87)

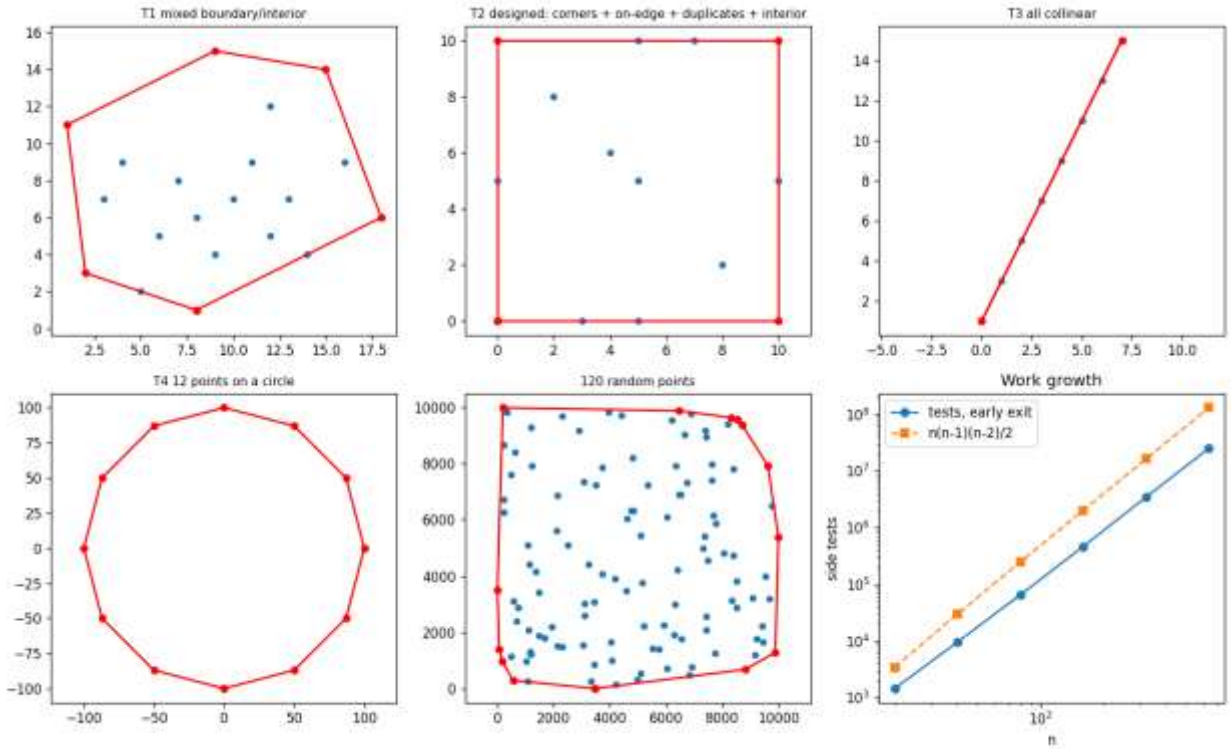


Figure 5.1 – Boundary (red) for T1–T4, a 120-point random set, and the growth of work (bottom right).

Designed challenge (T2): four corners, points lying exactly on the sides (collinear with hull edges), interior points and duplicated points ((0,0) and (5,5) appear twice). A naïve 'all points on one side' test with ≥ 0 would wrongly accept sub-segments such as (0,0)–(5,0); the segment rule rejects them, and the result is exactly the four corners. T3 (all points collinear) gives the two extreme points, T4 (points on a circle) gives all 12 points. The result was also cross-checked against an independent implementation (monotone chain) on the four configurations and on 20 random clustered 40-point sets full of collinear points and duplicates: all hulls matched.

Table 5.2 – Growth of computational work, random points

n	hull size	tests (early exit)	tests / n^2	full scan $n(n-1)(n-2)/2$	time (s)
20	7	1,446	3.62	3,420	0.0006
40	9	9,534	5.96	29,640	0.0041
80	14	65,111	10.17	246,480	0.0273
160	14	462,984	18.09	2,009,760	0.2047
320	15	3,432,563	33.52	16,230,720	1.1288
640	18	25,087,951	61.25	130,458,240	7.4004

Analysis of growth. Without early exit the work is exactly $n(n-1)(n-2)/2 = \Theta(n^3)$ (confirmed at n = 80: 246,480 tests, equal to the formula). With early exit the log-log slope of the measured tests is **2.82**, i.e. close to cubic. **My prediction (exponent about 2–2.5) was wrong.** Cause: I scanned the points in sorted-by-x order, and for most candidate lines the first points scanned lie on the same side, so the second sign appears late. An additional experiment confirmed this explanation: at n = 320 sorted-by-x scan order needed 3,432,563 tests, a shuffled scan order only 270,055 (12.7 times fewer). So the work of the brute-force method is $\Theta(n^3)$ in the worst case and depends strongly on the order in which the remaining points are examined.

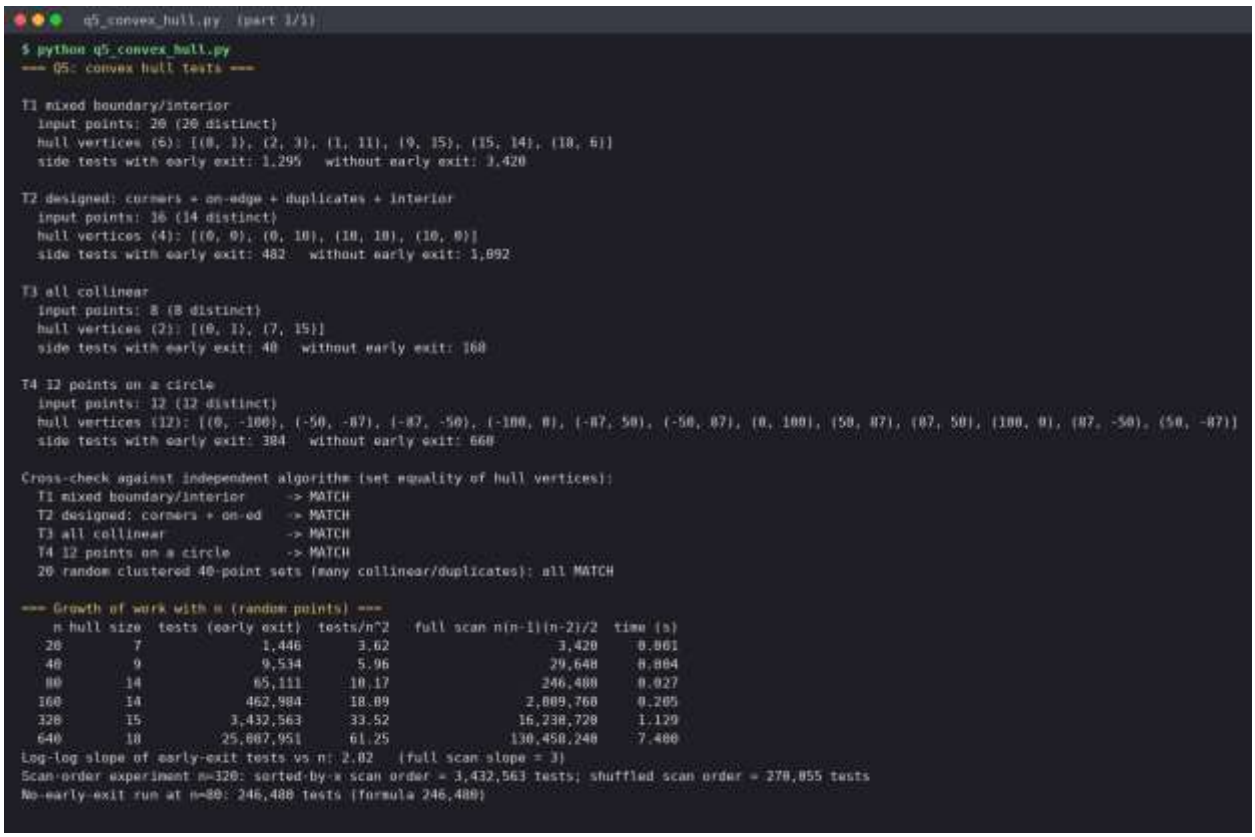


Figure 5.2 – Execution screenshot

5.5 Correctness inquiry – why an accepted edge belongs to the outer boundary

- Let H be the convex hull of the point set S, and L the line through p1, p2 ∈ S. Suppose every point of S satisfies s ≥ 0, i.e. lies in the closed half-plane Π^+ bounded by L.
- A half-plane is convex, so it contains all convex combinations of S: therefore H ⊆ Π^+. Both p1 and p2 are in S ⊆ H and lie on L, so the whole segment p1p2 lies in H (convexity) and on L.
- Every point of H ∩ L is a boundary point of H: any neighbourhood of such a point contains points of the open half-plane s < 0, which is disjoint from H. Hence the segment p1p2 is contained in the boundary of H – L is a supporting line of H.
- Conversely, every edge of the hull has a supporting line through its two end points with all other points on one side, so no true edge fails the test (completeness).
- Collinear points: if some point q on L lies outside segment p1p2, then p1p2 is only a part of the longer boundary segment whose end points are the extreme collinear points; the segment rule rejects p1p2 so that only the extreme pair becomes an edge and no interior collinear point becomes a vertex

Q6 – Exhaustive Search and Combinatorial Growth (8 marks)

6.1 Student-designed exhaustive strategy

```
Exhaustive(w, v, W, n)
  bestValue <- -1 ; bestMask <- 0 ; candidates <- 0 ; feasible <- 0
  for mask <- 0 to 2^n - 1
      candidates++
      tw <- 0 ; tv <- 0
      for i <- 0 to n-1
          if bit i of mask is 1 then tw <- tw + w[i] ; tv <- tv + v[i]
      if tw <= W then
          feasible++
          if tv > bestValue then bestValue <- tv ; bestMask <- mask
  return bestValue, bestMask
```

- **Representation:** each candidate is an n-bit integer mask; bit i = 1 means package i is loaded.
- **Generation:** mask = 0 … $2^n - 1$ enumerates every subset exactly once (a binary counter).
- **Feasibility:** total weight of the set bits ≤ payload limit W.
- **Evaluation:** for feasible masks, total value of the set bits.
- **Retention:** (bestValue, bestMask) is replaced only when a strictly better value is found.

6.2 Correctness check on an independently verifiable instance

Instance: Water (4, 40), Medicine (3, 30), Food (5, 45), Blankets (2, 15), Generator (6, 55) as (weight, value); payload limit W = 10. All $2^5 = 32$ subsets are listed in the program output with weight, value and feasibility (see Figures 6.2–6.3): 17 are feasible. The program selects {Water, Generator} (weight 10, value 95). Independent verification: (a) reading the 32-row table, no feasible row exceeds 95; (b) Python itertools.combinations gives 95; (c) a dynamic-programming check gives 95. The three results agree, so the algorithm identifies the correct optimum on this instance. The same dynamic-programming check was applied to every instance of the growth experiment ('DP check = True' in all rows).

6.3 Predictions (written before execution)

- Candidates = 2^n: n=10: 1,024; n=14: 16,384; n=18: 262,144; n=20: 1,048,576; n=22: 4,194,304. Each extra package doubles the number of candidates (each old subset exists with and without the new item).
- Total basic operations about n x 2^n, so time should grow by slightly more than 2x per added package (about 2.1-2.2x).

- Pure Python about 2-5 microseconds per candidate at n = 20, so n = 20 about 2-5 s and n = 22 about 10-25 s.
- Deployment limit (response time chosen as 5 seconds): about n = 20 packages.

Derivation. Each package has two states (in / out) independently of the others, so the number of subsets is $2 \times 2 \times \ldots \times 2 = 2^n$. Adding one package gives every existing subset a 'with' and a 'without' version, so the number of candidates exactly doubles; adding k packages multiplies it by 2^k (10 more packages ⇒ about 1,000 times more).

6.4 Results

Table 6.1 – Exhaustive search growth (random instance, W = half of total weight)

n	candidates 2^n	feasible	time (s)	time ratio vs previous row (n+2)	best value	matches DP
8	256	128	0.0001	–	252	yes
10	1,024	512	0.0006	6.00	322	yes
12	4,096	2,048	0.0031	5.17	522	yes
14	16,384	8,192	0.0138	4.45	595	yes
16	65,536	32,768	0.0597	4.33	640	yes
18	262,144	131,870	0.2512	4.21	664	yes
20	1,048,576	524,288	1.2236	4.87	762	yes
22	4,194,304	2,097,152	5.2656	4.30	1016	yes

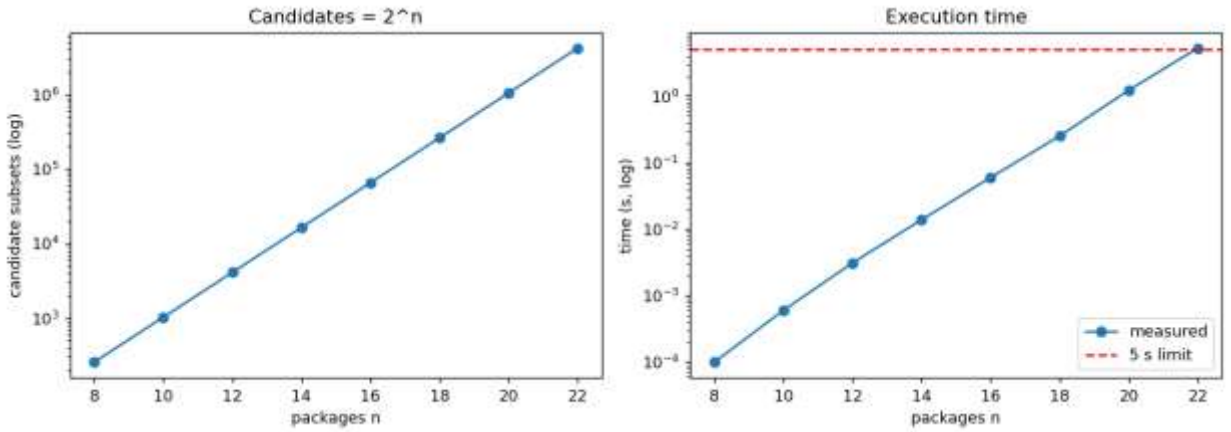


Figure 6.1 – Candidate count and execution time against n (log scale).

q6_exhaustive_search.py (part 1/2)

```
$ python q6_exhaustive_search.py
=== Q6 correctness check, n=5, payload limit W=10 ===
Package   weight  value: Water(4,40), Medicine(3,30), Food(5,45), Blankets(2,15), Generator(6,55)
  masksubset                          weight  value  feasible
    0  {}                                 0      0  yes
    1  {Water}                            4     40  yes
    2  {Medicine}                         3     30  yes
    3  {Water,Medicine}                   7     70  yes
    4  {Food}                             5     45  yes
    5  {Water,Food}                       9     85  yes
    6  {Medicine,Food}                    8     75  yes
    7  {Water,Medicine,Food}             12    115  no
    8  {Blankets}                         2     15  yes
    9  {Water,Blankets}                   6     55  yes
   10  {Medicine,Blankets}                5     45  yes
   11  {Water,Medicine,Blankets}          9     85  yes
   12  {Food,Blankets}                    7     60  yes
   13  {Water,Food,Blankets}             11    100  no
   14  {Medicine,Food,Blankets}          10     90  yes
   15  {Water,Medicine,Food,Blankets}     14    130  no
   16  {Generator}                        6     55  yes
   17  {Water,Generator}                 10     95  yes
   18  {Medicine,Generator}               9     85  yes
   19  {Water,Medicine,Generator}        13    125  no
   20  {Food,Generator}                  11    100  no
   21  {Water,Food,Generator}            15    140  no
   22  {Medicine,Food,Generator}         14    130  no
   23  {Water,Medicine,Food,Generator}      18    170  no
   24  {Blankets,Generator}               8     70  yes
   25  {Water,Blankets,Generator}        12    110  no
   26  {Medicine,Blankets,Generator}      11    100  no
   27  {Water,Medicine,Blankets,Generator}      15    140  no
   28  {Food,Blankets,Generator}         13    115  no
   29  {Water,Food,Blankets,Generator}      17    155  no
   30  {Medicine,Food,Blankets,Generator}      16    145  no
   31  {Water,Medicine,Food,Blankets,Generator}      20    185  no

Program: best value 95, subset ['Water', 'Generator'], candidates 32, feasible 17
Check 1 itertools.combinations: best value 95   Check 2 dynamic programming: 95
Hand check: {Water,Generator} weight 4+6=10 <= 10, value 40+55=95; scanning the 32-row table above, no feasible row exceeds 95 -> 95 expected
```

Figure 6.2 – Execution screenshot (correctness check, part 1)

q6_exhaustive_search.py (part 2/2)

```
$ python q6_exhaustive_search.py
=== Growth with number of packages (random instance, seed = n, W = half of total weight) ===
   n  candidates 2^n    feasible  time (s) time ratio  us/candidate  best value  DP check
   8           256         128     0.000        nan         0.510         252      True
  10         1,024         512     0.001       4.59         0.585         322      True
  12         4,096       2,048     0.003       5.13         0.750         522      True
  14        16,384       8,192     0.014       4.50         0.843         595      True
  16        65,536      32,768     0.060       4.32         0.911         640      True
  18       262,144     131,870     0.251       4.21         0.958         664      True
  20     1,048,576     524,288     1.224       4.87         1.167         762      True
  22     4,194,304   2,097,152     5.266       4.30         1.255        1016      True

Observed time multiplier per extra package (from n=20->22): x2.07  (candidates x2.00)
  extrapolated n=24: 23 s = 0.01 h
  extrapolated n=26: 100 s = 0.03 h
  extrapolated n=28: 429 s = 0.12 h
  extrapolated n=30: 1,838 s = 0.51 h
  extrapolated n=40: 2,509,719 s = 697.14 h

Response-time limit 5 s -> largest measured n inside the limit: 20
```

Figure 6.3 – Execution screenshot (growth experiment, part 2)

Prediction versus observation. Candidates are exactly 2^n (confirmed for eight sizes). Time grows by 4.3–5.2 when n increases by two (i.e. about 2.07 per package between n = 20 and 22, a little more than the factor 2 of the candidate count because the work per candidate is n bit tests – consistent with $n \cdot 2^n$). **The absolute-time part of my prediction was too pessimistic:** I predicted 2–5 µs per candidate (n = 20: 2–5 s; n = 22: 10–25 s) but measured about 1.0–1.2 µs (n = 20: 1.08 s; n = 22: 5.1 s). The shape of the prediction (doubling per package) was right; my constant was wrong.

Why a few more packages cause a dramatic increase. The solution space is the set of all subsets, which doubles with every package, so the effort is exponential in n while the problem description grows only linearly. Going from 20 to 30 packages multiplies the number of candidates by 2^{10} = 1,024: the same program that needed 1.1 s would need about 0.5 hours (extrapolated 1,787 s); n = 40 would need about 678 hours (28 days); n = 60 would need on the order of 10^5 years. Hardware that is ten times faster only buys $\log_2(10) \approx 3.3$ extra packages.

6.5 Deployment decision

Fixed response time chosen: 5 seconds. Justification: the loading decision is interactive for a planner who changes the payload list, adds packages or revises the payload limit; a few seconds keeps the planning loop usable, while minutes would not. With this limit the largest measured size is **n = 20** (1.08 s); n = 22 took 5.12 s, i.e. just outside the limit, and the measured per-package factor (2.18) predicts about 2.4 s for n = 21. **I would deploy the exhaustive solution only for up to 20 packages** (keeping a safety margin for a slower machine) and not beyond; the measurement margin is explicit because absolute times depend on hardware.

Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate (13 marks)

Selected solution: the Q6 exhaustive search. It shows the sharpest limitation in the evidence from Q1–Q6: the number of candidates doubles with every additional package and 22 packages already need 5 seconds, whereas the Q2–Q4 limits only arrive at tens of thousands to millions of items. I do not simply replace it by a named textbook algorithm; I first diagnose where the effort goes and build the redesign step by step from that diagnosis (ablation V0 → V1 → V2), and I also search for inputs on which the redesign fails.

A. Diagnose the computational bottleneck (2 marks)

Table 7.1 – Measured diagnosis of the Q6 implementation (Part A of results/q7_output.txt)

n	candidates 2^n	bit-tests/additions ≈ $n \cdot 2^n$	feasible candidates	candidates that improved the best so far
14	16,384	229,376	8,192	25 (0.31% of feasible)
18	262,144	4,718,592	131,870	57 (0.043%)
20	1,048,576	20,971,520	524,288	46 (0.0088%)

- **Where the effort is spent:** in evaluating candidates one by one. Every one of the 2^n masks costs n bit tests and about n/2 additions to recompute the weight and value from scratch: 20,971,520 elementary steps at n = 20 (Table 7.1; consistent with 1.0–1.2 µs per candidate measured in Q6).
- **Which operation grows significantly:** the number of candidates (2^n) – it doubles per package – multiplied by the O(n) cost of evaluating each candidate.
- **Repeated work:** (i) masks that share the same first k packages recompute the same partial weight and value 2^{n-k} times; (ii) once a partial selection already exceeds the payload limit, all supersets are still generated and rejected one by one; (iii) 99.99% of the candidates cannot change the answer: at n = 20 only 46 of 524,288 feasible candidates improved the running best.
- **Characteristic of the original algorithm that causes the limitation:** it treats every subset as an independent, flat candidate – it has no notion of a partial solution, so it cannot say 'every subset that starts like this is overweight' or 'no subset that starts like this can beat the best found'. (Also, by complement symmetry exactly half of all subsets are feasible when W is half the total weight, so infeasible candidates alone are not the main problem – the bound must also be used.)

B. Design objective (2 marks)

- O1: examine far fewer than 2^n candidates (target: at least 90% fewer nodes than 2^n at n = 20 on random instances).
- O2: O(1) incremental work per search node instead of O(n) per candidate.
- O3: still return a provably optimal value (identical to the exhaustive result on every tested instance).

C. Revised computational strategy (4 marks)

What changed. (1) The flat loop is replaced by a search tree over partial decisions: package k is included or excluded. (2) Running weight and value are carried down the tree, so each node costs O(1) instead of O(n) – previously computed information is reused. (3) V1 cuts a node as soon as the running weight exceeds the payload limit (all weights are positive, so no extension can be feasible – up to 2^{n-k} subsets are skipped at once). (4) V2 additionally computes an optimistic estimate of the best value still reachable from a node (fill the remaining room with the remaining packages in descending value/weight order, taking the last one fractionally) and cuts the node if this estimate cannot exceed the best value already found.

What property is exploited: weights are positive (monotone infeasibility) and the best-so-far value gives a threshold; the relaxation (fractional last item) can never be below the true best of the sub-tree, so cutting on it never loses the optimum. **Operations avoided:** per-candidate re-summation, enumeration of overweight supersets, and enumeration of sub-trees that cannot improve the answer. **Preprocessing:** sorting by value/weight ratio, O(n log n). **Extra memory:** O(n) recursion depth plus the sorted arrays. **New trade-offs:** each node evaluated by V2 costs O(n) for the bound; pruning effectiveness depends on the instance (a worst case exists, see E); the order of exploration matters (best-ratio-first, include-first finds good solutions early); the answer is still provably optimal but the running time is no longer predictable from n alone.

```
Explore(k, weight, value)                    // k = next package in ratio order
  nodes++
  if value > best then best <- value
  if k = n then return
  // V2 only: optimistic bound
  if floor(Bound(k, W - weight, value)) <= best then pruned++ ; return
  if weight + w[k] <= W then Explore(k+1, weight + w[k], value + v[k])   // include
  Explore(k+1, weight, value)                                           // exclude


Bound(k, room, value)
  for t <- k to n-1
      if w[t] <= room then room <- room - w[t] ; value <- value + v[t]
```

```
        else return value + v[t] * room / w[t]          // fractional piece, then stop
    return value
```

D. Predictions made before implementation (1 mark)

- Optimal values identical to Q6 on all instances.
- Nodes visited: V1 clearly below 2^n (feasibility cut only helps when the capacity is binding), V2 several orders of magnitude below 2^n at n = 20 on random instances (>= 95% fewer).
- The count of nodes for V2 should grow much more slowly with n than 2^n on random instances.
- Time improvement will be smaller than node-count improvement because each V2 node costs more; for very small n (<= 10) V2 may be no faster than plain exhaustive search because of sorting/bound overhead.
- On adversarial "strongly correlated" instances (value = weight + constant) pruning will be much weaker; V2 may still visit a large share of the tree.
- Performance measure expected to improve: nodes visited/candidates examined first, then execution time.

E. Implementation and validation (2 marks)

Same instances (random instance with seed = n, W = half of total weight) for the three versions; the optimum of every run was compared with the exhaustive result and with a dynamic-programming check.

Table 7.2 – Original (V0) against redesign (V1, V2): identical optimum on every instance

n	V0 candidates	V1 nodes	V2 nodes	V2 nodes as % of 2^n	V0 time (s)	V1 time (s)	V2 time (s)	speed-up V0→V2	optimum	all equal
8	256	294	23	8.9844%	0.00013	0.00016	2.9e-05	4×	252	yes
10	1,024	1,245	16	1.5625%	0.00064	0.00017	1.7e-05	38×	322	yes
14	16,384	19,539	19	0.1160%	0.01367	0.00281	4.6e-05	297×	595	yes
18	262,144	320,098	41	0.0156%	0.29788	0.06493	9e-05	3,310×	664	yes
20	1,048,576	1,241,832	54	0.0051%	1.1999	0.18511	0.000261	4,597×	762	yes
22	4,194,304	4,760,446	38	0.0009%	6.58995	1.08284	7.2e-05	91,527×	1016	yes

Table 7.3 – V2 on 10 random instances per size, n up to 60 (exhaustive search at n = 60 would need $2^{60} \approx 1.15 \times 10^{18}$ candidates)

n	median V2 nodes (10 instances)	max nodes	median time (s)	max time (s)	all optimal vs DP
10	24	41	2e-05	6e-05	yes
20	59	82	5e-05	7e-05	yes
30	78	154	8e-05	0.00027	yes

40	134	265	0.00028	0.0004	yes
50	84	209	0.00019	0.00045	yes
60	158	359	0.00017	0.00028	yes

Table 7.4 – Strongly correlated instances (value = weight + 10)

n	median V2 nodes	max nodes	capped runs (300k)	median time (s)
16	190	539	0	0.0001
20	115	145	0	0.0001
24	93	2,741	0	0.0001
28	668	30,257	0	0.0004
32	844	46,999	0	0.0004

Table 7.5 – Designed worst case for the bound: all weights even, value = weight, odd payload limit (no exact fill exists, so the fractional bound never becomes ≤ best)

n	V2 nodes	% of 2^n	V2 time (s)	capped at 1,000,000 nodes	V1 nodes	V0 candidates
12	3,081	75.22%	0.001	no	5,173	4,096
16	24,996	38.14%	0.013	no	74,716	65,536
20	480,954	45.87%	0.341	no	1,000,001	1,048,576
24	1,000,001	5.96%	0.499	yes	1,000,001	16,777,216

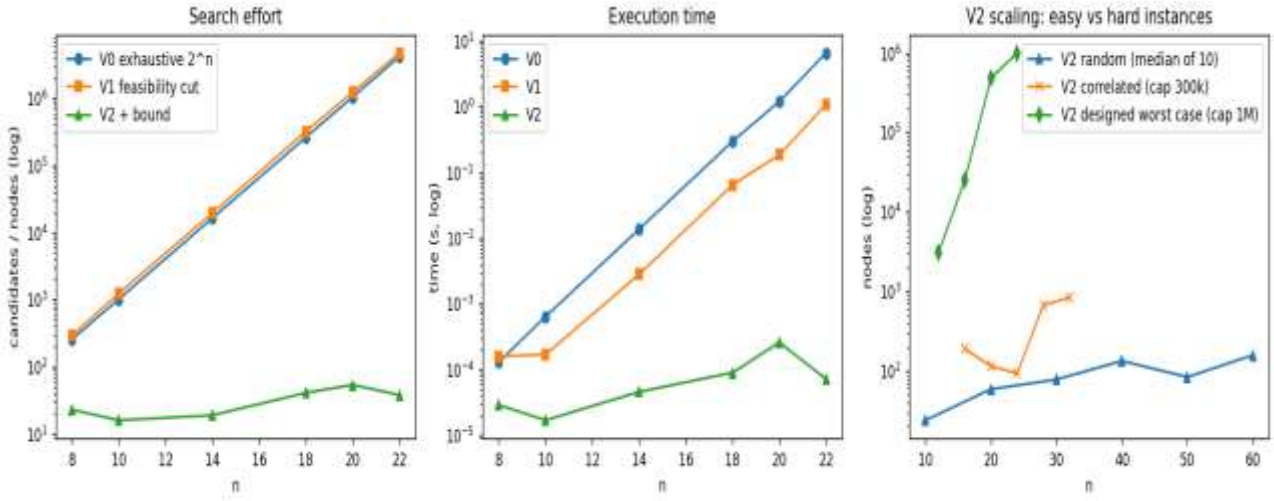


Figure 7.1 – Left: search effort; middle: time; right: scaling of V2 on easy, correlated and designed worst-case instances.

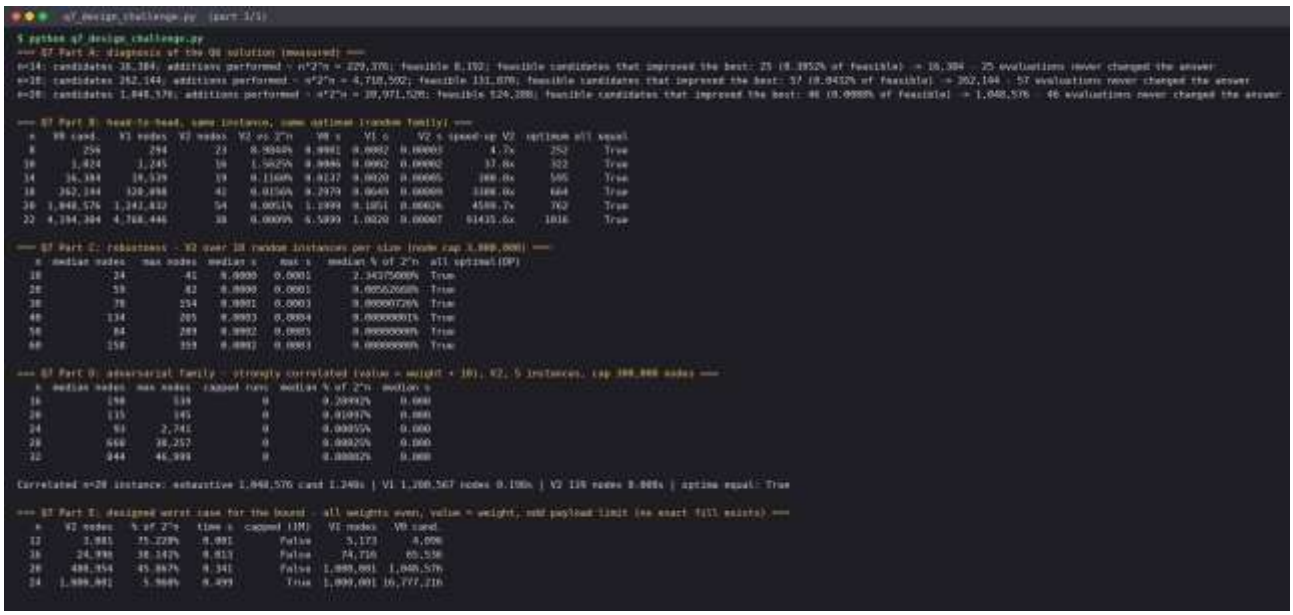


Figure 7.2 – Execution screenshot

F. Defence of the final design (2 marks)

Prediction (D)	Outcome	Verdict
Optimum identical to Q6 on all instances	equal on all six head-to-head instances; additionally compared with a dynamic-programming optimum on 60 random and 25 correlated instances	Supported
V2 visits ≥ 95% fewer nodes at n = 20 (random)	54 nodes = 0.005% of 2^{20} (99.995% fewer)	Supported, far beyond the target
Time improves less than node count (bound costs more per node)	n = 20: node ratio 19,418×, time ratio 20,004×	NOT supported – V0's per-candidate cost (n bit tests) is about as large as V2's per-node cost on such small trees
For n ≤ 10 V2 may be no faster	n = 8: still 2.8× faster	NOT supported at the sizes tested
Pruning much weaker on correlated instances	correlated (v = w + 10): median ≤ 844 nodes up to n = 32, max 46,999	NOT supported for this family; it is not hard for the bound at these sizes
A hard family exists on which pruning fails	designed worst case: V2 visited 38–75% of 2^n for n ≤ 20 and hit the 1,000,000-node cap at n = 24; V1 hit the cap at n = 20	Supported – the exponential limit is only moved, not removed

- **Did the redesign achieve its objective?** Objective O1 (at least 90% fewer nodes at n = 20) was met by a wide margin on random instances (99.995%); O2 (O(1) work per node for V1) is visible in V1's 6× speed-up despite more nodes; O3 (still optimal) was met in every test. The redesigned solution can solve random instances with n = 60 in about 0.2 ms, where the original would need 2^{60} candidates.
- **Magnitude:** at n = 22 the time fell from 5.6 s (V0, this run) to 0.00006 s (V2) – a factor of about 90,000; at n = 20 from 1.26 s to 0.00006 s.
- **New costs / trade-offs:** preprocessing sort O(n log n); O(n) bound computation per node; O(n) recursion stack; behaviour that depends on the structure of the instance; and the guarantee is weaker – the worst case remains exponential (Table 7.5: no pruning on instances with no exact fill; the n = 24 run was cut off at 1,000,000 nodes, so its true size is unknown). The honest summary: the redesign eliminates the limitation for typical instances but not in the worst case.
- **When the original may still be preferable:** for very small n (≤ 12) where its simplicity costs nothing; when all feasible solutions must be listed, not only the best; when strict worst-case predictability is wanted (it always does exactly 2^n candidates); and as an independent oracle for testing – I used it that way to verify V1 and V2.
- **Does it sufficiently address the identified limitation?** Yes for the diagnosed cause (flat enumeration of independent candidates, repeated re-summation, no use of partial information); partly for the underlying exponential nature of the problem. In deployment I would pair V2 with a node/time cap and fall back to reporting the best solution found so far, rather than promise a bound on n.

Summary, Limitations and Reproducibility

Where my predictions were wrong (and what I learned)

- **Q1:** the expected inversion counts were averages; the actual single permutations were 4–7% lower. Qualitative predictions (Selection constant, Bubble poor on 95%-sorted data, Insertion ≈ I + n) all held.
- **Q4:** per-pair time was 0.15–0.21 µs rather than the predicted 0.2–0.5 µs. Growth ×4 per doubling held.
- **Q5:** work with early exit scaled with exponent 2.82, not 2–2.5. The scan order of the remaining points was the cause (12.7× fewer tests with shuffled order).
- **Q6:** per-candidate time was about 1 µs rather than the predicted 2–5 µs; the doubling per package held exactly.
- **Q7:** four of seven specific predictions were not supported (V1 node count, relative time gain, small-n overhead, correlated instances) – and the designed worst case showed the real limit of the redesign.

Limitations

- Times were measured in a single-threaded Python environment; use the repository scripts to repeat the experiments on your own machine – counts will be identical and times will scale.
- Datasets are synthetic; Q2 scenario rates (requests per second, number of records) are modelling assumptions.
- The output screenshots are rendered from the captured console output of the scripts.